SWI (Soft Web Intelligence) Powered by User-Defined Fuzzy Operators and Aggregators

Paolo Fosci and Giuseppe Psaila(✉)

DIGIP, University of Bergamo, Viale Marconi 5, 24044 Dalmine, BG, Italy
{paolo.fosci,giuseppe.psaila}@unibg.it

Abstract. We proposed the notion of Soft Web Intelligence in our previous works: in short, it interprets the general notion of Web Intelligence in a modern key, by explicitly considering the current technological panorama. Specifically, *JSON* data sets are acquired from the Internet, stored within *JSON* document stores and then processed and queried by means of soft computing and soft querying methods.

In this paper, we present the joint adoption of fuzzy operators and fuzzy aggregators to perform data-analysis tasks on aggregated data. Indeed, when data are processed through fuzzy sets to perform soft querying, data items with membership degrees to fuzzy sets could be aggregated, to obtain a novel membership to another fuzzy set belonging to a different universe; in this context, User-defined Fuzzy aggregators become critical for actually performing Soft Web Intelligence based on soft computing.

Keywords: Soft web intelligence · User-defined fuzzy operators and fuzzy aggregators · Novel constructs in J-CO-QL+ · Soft querying on JSON data sets

1 Introduction

After two decades, the concept of Web Intelligence is still a hot topic of research in computer science. The paper [23] proposed Web Intelligence for the first time: the idea is to move from the well-known approaches that fall under the hat of Business Intelligence (or BI), so as to adapt them for gathering and analyzing possibly semi-structured data sets from the Word-Wide Web, so as to discover useful knowledge from Web sites and Web sources. In [23], the authors argued that Artificial Intelligence (or AI) is the key technology to get rid of the complexity of the Web, in terms of variety and complexity of formats which data must be extracted from.

Among AI techniques, "soft computing" and "fuzzy logic" have played a significant role, due to their capabilities of easily dealing with imprecision and vagueness both

This study was funded by the European Union - *NextGenerationEU, in the framework of the GRINS – Growing Resilient, INclusive and Sustainable project (GRINS PE00000018 - CUP F83C22001720001)*. The views and opinions expressed are solely those of the authors and do not necessarily reflect those of the European Union, nor can the European Union be held responsible for them.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2025
M. Marchiori and F. García Peñalvo (Eds.): WEBIST 2023, LNBIP 543, pp. 77–100, 2025.
https://doi.org/10.1007/978-3-031-89621-7_5

within data and within queries over data. We demonstrated these capabilities in our recent research works [5,7,12], whose goal is to develop a software framework, named *J-CO* Framework, which integrated advanced capabilities of managing and integrating large collections of *JSON* documents with sophisticated capabilities of soft querying *JSON* documents through fuzzy sets.

As far as Web Intelligence is concerned, in our recent papers [7,9,10] we introduced and explored the notion of *Soft Web Intelligence*, i.e., a precise and focused interpretation of the general notion of Web Intelligence, in which the *J-CO* Framework plays a key role, so as to exploit its capabilities of soft querying o actually extract useful information from Web source that provides *JSON* data sets.

In *Soft Web Intelligence*, user-defined fuzzy operators and fuzzy aggregators are the key linguistic tools to perform soft queries on *JSON* data sets acquired from Web sources. The main contribution of this paper is to unify previous contributions to the exploitation of either fuzzy operators or of fuzzy aggregators into a homogeneous framework; this way, we show the potential of applying complex soft processes executed by the *J-CO* Framework towards performing tasks of *soft Web Intelligence*.

The paper is organized as follows. Section 2 presents the background of our work as far as related work is concerned. Section 3 introduces the vision of *Soft Web Intelligence*. Section 4 presents the main features of the *J-CO* Framework (in Sect. 4.1) and how fuzzy sets are supported in the *J-CO-QL*$^+$ language (in Sect. 4.2). Section 5 introduces the novel construct that we recently added to *J-CO-QL*$^+$ to declare user-defined fuzzy aggregators, together with the semantic model. Section 6 addresses a practical case study by exploiting user-defined fuzzy aggregators. Finally, Sect. 7 draws conclusions and future work.

2 Related Work

In this section, we present the background from which this work has originated.

The concept of Web Intelligence was introduced in [23]. It can be conceived as the evolution of "Business Intelligence" [17] towards the World Wide Web; "Artificial Intelligence" (AI for short) was supposed to be the key tool for performing Web Intelligence, but what is Artificial Intelligence?

In the current common perception, AI means "Neural Networks" and "Deep Learning", due to the success they have recently obtained. Nevertheless, techniques such for "Data Mining" belongs to AI too and are very popular, so they have been successfully applied to Web Intelligence [14,29].

Nonetheless, in the literature, many papers try to give a more specific yet wider interpretation of Web Intelligence, such as "Computational Web Intelligence" [27], i.e., the adoption of "Computational Intelligence" in Web Intelligence, as well as "Brain Informatics" [28], i.e., fostering Web Intelligence through techniques that come out from the study on the human brain.

Fuzzy Logic and Soft Computing belong to AI too: indeed, their capabilities of approximate reasoning based on "linguistic predicates" provides a significant contribution towards AI.

Remember that a Fuzzy Set A in a universe U is a mapping for each $x \in U$, $A : x \rightarrow [0, 1]$, also denoted as $\mu_A : x \rightarrow [0, 1]$. The co-domain is the set of "membership

degrees" (or simply "memberships", for brevity in the following): each item x belongs to A with a degree; when the degree is $0 < \mu_A(x) < 1$, x belongs to A only partially; obviously, $\mu_A(x) = 1$ denotes full membership of x to A, while $\mu_A(x) = 0$ means that x does not belong at all to A.

If fuzzy sets represent either approximate or imprecise properties that are easily synthesized by "linguistic terms" (e.g., "young people", "rich people", and so on), then selection predicates become "linguistic predicates", corresponding to fuzzy sets. Therefore, the membership degree of an item to a fuzzy set denotes how the item satisfies the represented linguistic predicate.

The reader can guess that fuzzy logic can be exploited for Web Intelligence. In this regard, Zadeh, the creator of Fuzzy-Set Theory [24], in [25,26] proposed the concept of "WebIQ" or "WIQ", i.e., a formal framework in which Fuzzy-Set Theory provides the formal tools to query Web sources in an extensive and imprecise way.

Nevertheless, when performing Data Intelligence activities [1] data must be aggregated; if we use linguistic predicates and fuzzy logic, "fuzzy aggregations" become essential. We are focused on groups (or categories) $G_j = \{x_{j,1}, x_{j,2}, \dots\})$ of items $x_{j,i}$ that belong to the same group G_j because they share some common properties or are samples of the same category of items. Each $x_{j,i}$ singularly may belong to a fuzzy set A, thus it is provided with a membership $\mu_A(x_{j,i})$. Consequently, the fuzzy set $\overline{A}$ of groups G_j can be seen as a partition of A: the membership degree of a group G_j to $\overline{A}$ should be derived by somehow aggregating memberships $\mu_A(x_{j,i})$, group by group. Alternatively, if $x_{j,i}$ has not a membership, its values might have to be aggregated to obtain the final membership of the G_j group.

Popular fuzzy aggregators of this type are "Weighted aggregation" (see [3]) and "Ordered Weighted Aggregation" (OWA) [16,22].

As a final remark for this section, we want the reader to be aware that very few papers are available in the literature that talk about fuzzy logic and soft computing in Web Intelligence. We found the paper [15]: it exploits soft computing in a group decision-making system to express preferences, but Web Intelligence activities were not supported by soft computing. We also found the paper [18]: it uses `FUZZYALGOL`, a fuzzy procedural programming language [21], for soft querying Web sources.

3 Soft Web Intelligence

In [9], we conceived the notion of *Soft Web Intelligence* as an interpretation of the general notion of Web Intelligence on the basis of the current technological panorama. Indeed, the current technological panorama has been characterized by the advent of *JSON* as *de-facto* standard format for information interchange over the Internet, in place of XML. One important consequence of this phenomenon has been the development of a specific category of *NoSQL* databases (i.e., databases that are not relational and do not use SQL as query language) known as "*JSON* document stores", because they directly manage collections of *JSON* documents; *MongoDB* appears to be the most popular one.

Hereafter, we present our vision of *Soft Web Intelligence*, by relying on Definition 1 (which comes from our previous work [10]).

Definition 1. *"Soft Web Intelligence" is the continuous acquisition, integration and querying of JSON data sets coming from or representing Web sources, by exploiting*

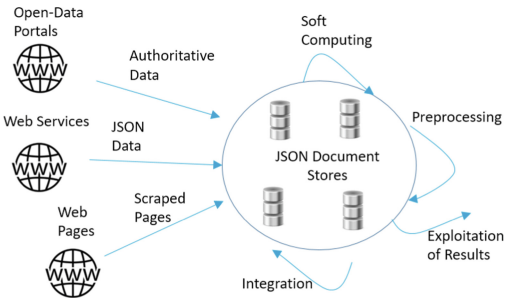


Fig. 1. Vision of Web Intelligence.

Soft Computing and Soft Querying, to use them for decision-making and knowledge discovery.

As the reader can see, Definition 1 instantiates the general concept of *Web Intelligence* (provided more than 20 years ago [23]) by considering the current technological panorama.

Through Fig. 1, we illustrate how we figure out *Soft Web Intelligence*.

- **Web Sources.** Web technology has significantly evolved in the last two decades. Consequently, many different types of Web sources are now available, apart from classical Web sites. Here, we focus on Web sources that provide *JSON* data sets.
  - Web Services plays a key role in the modern Web panorama: they are expected to provide either complete data sets or single pieces of data. If the Web service works only in this latter mode and many piece of information must be acquired, this means that the full data set of interested must be built by singularly collecting all data items. Typically, social media expose Web services that can be exploited to interact with the system; nowadays, *JSON* is the data format on which most of Web services rely.
  - Common channels that public administrations exploit to publish data sets (often referred to as "Authoritative Data Sets") concerning the administered territory are Open-Data portals. Among all formats, *JSON* is becoming more and more popular in this context too.
  - The content of Web pages (i.e., HTML pages) could be seen as a giant repository of precious information. Techniques for "Web Scraping" could acquire the content of HTML pages and represent it as *JSON* data sets.
- ***JSON* Document Stores.** Data acquired from Web sources as *JSON* data sets should be stored within a pool of databases managed by (potentially different) *JSON* document stores. Currently, *MongoDB* is (in our perception) the most popular *JSON* store. Nevertheless, other *JSON* stores are available and widely diffused: for example, *CouchDB* has been chosen as the DBMS for the *HyperLedger Fabric* permissioned BlockChain platform [2]).
- **Processing Activities.** Processing is the key factor for the success of *Soft Web Intelligence*.

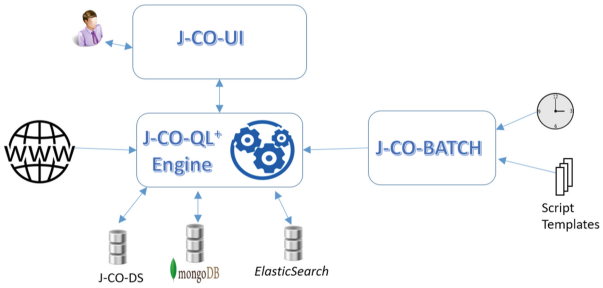


Fig. 2. The *J-CO* Framework.

- The first processing activity to perform is "Preprocessing": indeed, it is necessary to remove noise from data and make their formats homogeneous.
- The second activity to perform on data is "Integration": indeed, the different data sets (possibly stored in different databases) must be integrated, so as to unite those pieces of information that came from different sources.
- The final activity to perform on data sets is "Soft Computing": during this activity, techniques based on soft computing and soft querying are used to extract knowledge from integrated data.

Notice that the tool that provides soft computing capabilities strongly determines the level of complexity that the analysis tasks can reach. This is why we are introducing fuzzy aggregators within the framework we used to develop the concept of *Soft Web Intelligence*, as it will be illustrated in the next sections.

4 Fuzzy Sets in the J-CO Framework

In this section, we present the *J-CO* Framework and how its language (named *J-CO-QL*$^+$) provides users with constructs to define fuzzy operators, to perform complex soft queries on *JSON* data sets. Then, in Sect. 5 we will extensively present the novel concept of "fuzzy aggregator".

4.1 The J-CO Framework

The *J-CO* Framework is a pool of software tools whose goal is to provide analysts with a powerful support for gathering, integrating and querying possibly-large collections of *JSON* data sets. The core of the framework is its query language, named *J-CO-QL*$^+$.

The current organization of the framework, which is the result of [9], is depicted in Fig. 2. We explain it hereafter.

– *J-CO-QL*$^+$ *Engine*. This component actually executes *J-CO-QL*$^+$ scripts (i.e., queries). It is able to retrieve data from document databases (for example, managed by *MongoDB*) and saves results into them; it also can send HTTP requests to Web sources to get *JSON* data sets directly from them.

Listing 1. *J-CO-QL*$^+$: Fuzzy Operator `casePopulationFO`.

```
1. CREATE FUZZY OPERATOR casesPopulationFO
     PARAMETERS   cases  TYPE NUMERIC,
                  pop    TYPE NUMERIC
     PRECONDITION pop > 0 AND cases >= 0
     EVALUATE     100000 * cases / pop    // cases per 100k inhabitants
     POLYLINE     [ (  0, 0.0), (100, 0.1),
                    (300, 0.9), (900, 1.0) ];
```

– *J-CO-DS*. This component is a simple document store specifically introduced in [19] to store large or very large single *JSON* documents (such as many *GeoJSON* documents that cannot be stored within other *JSON* stores, such as *MongoDB*, which has a 16Mb limit per single document).
– *J-CO-BATCH* (introduced in [9]) is an off-line executor of *J-CO-QL*$^+$ scripts; in particular, it is possible to schedule the repeated execution of scripts. Another feature is the concept of "template": *J-CO-QL*$^+$ scripts can be parameterized to reuse them with different settings/configurations.
– *J-CO-UI* is the user interface, by means of which analysts can write *J-CO-QL*$^+$ scripts, submit them to the *J-CO-QL*$^+$ *Engine* and inspect results.

4.2 Fuzzy Sets and Operators

The *J-CO-QL*$^+$ language manipulates "collections" of *JSON* documents.

For each single document, it is possible to evaluate its memberships to many fuzzy sets in an independent way. These degrees are represented within the same document, by adding a special root-level field named `~fuzzysets`. It is a nested document that behaves as a key/value map: a field within it denotes the membership to a fuzzy set, in such a way the field name is the name of the eponym fuzzy set, while the value (in the range $[0, 1]$) is the membership degree.

In order to deal with fuzzy sets, *J-CO-QL*$^+$ provides constructs to derive membership degrees of *JSON* documents from their fields. Indeed, the membership degree of a document to a fuzzy set certainly depend on properties described by its field; through "user-defined fuzzy operators", it is possible to derive membership degrees of documents to fuzzy sets.

Consider Listing 1: it reports the definition of a user-defined fuzzy operator in the *J-CO-QL*$^+$ language. The goal of the operator is to provide a membership degree moving from two numerical parameters. Hereafter, we present it in details.

– The name of the fuzzy operator is `casesPopulationFO`. Its goal is to derive a membership degree as follows: given the number of cases of a disease and the number of inhabitants of a country, it computes the number of cases per 100,000 inhabitants and uses this value to obtain the membership degree.
– The clause `PARAMETERS` defines the formal parameters of the fuzzy operator. Specifically, two parameters are defined, i.e., the number of cases (the parameter `cases`) and the population (the parameter `pop`), both as integer values.

– The clause `PRECONDITION` specifies a Boolean condition that must be satisfied before evaluating the operator: if the condition is not satisfied, the operator is not evaluated and an error is raised. Specifically, the precondition says that the parameter `pop` must be greater than zero, while the parameter `cases` must be no less than zero.
– The clause `EVALUATE` computes the value that is later used to obtain the membership degree. It is a mathematical expression that exploits the parameters. Specifically, the number of cases for every 100,000 inhabitants is computed.
– The clause `POLYLINE` specifies the actual membership function.
J-CO-QL$^+$ provides users with the possibility to define polylines as membership function to express complex semantics. The polyline is defined by a sequence of points, in the form (x_i, y_i): x_i is any real number, while y_i is a real number in the range $[0, 1]$; given a point with index $i > 1$, $x_i >= x_{i-1}$.
If we denote the value provided by the `EVALUATE` clause with x, it is matched on the x-axis of the polyline, to obtain the corresponding value on the y-axis, as the corresponding membership degree. Note that if $x < x_1$ (respectively, $x > x_n$, with n the greatest i index), then y_0 (respectively, y_n), will be returned as membership degree.
Figure 3a depicts the polyline defined for the fuzzy operator under definition. The "S-shape" of the function allows for reducing the relevance (in terms of membership degree) for a number of cases that is less than 100 per 100,000 inhabitants; at the same time the shape quickly enhances the relevance when the cases grow up from 100 to 500 per 100,000 inhabitants; after 500, the membership slowly grows up from 0.9, reaching 1 for 900 cases per 100,000 inhabitants.

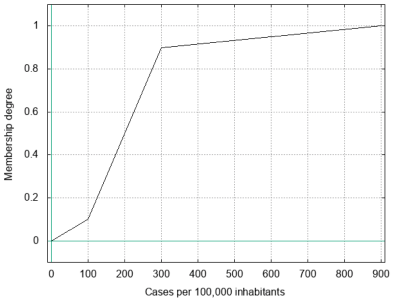
The statements of *J-CO-QL*$^+$ that manages *JSON* documents, provide specific clauses whose goal is to evaluate membership degrees of documents to fuzzy sets, so as to specify "soft queries" (in Sect. 6.1, we will show how to use fuzzy operators in practice).

In our previous research works, we explored the adoption of fuzzy sets for soft querying in many contexts. For example, in [7] we considered soft querying for spatial data sets, on the basis of complex geometries and spatial relationships between them. We also addressed the definition of a high-level domain-specific language named *GeoSoft* [4,12], whose execution relies on fuzzy capabilities of *J-CO-QL*$^+$ to process spatial data. Furthermore, we demonstrated that soft querying is effective to integrate geotagged data sets based on registry data and coordinates [7].

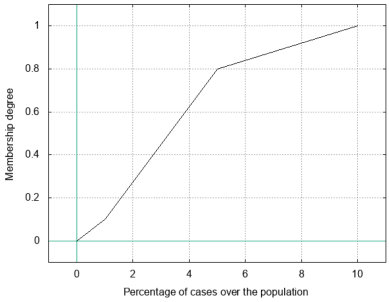
5 User-Defined Fuzzy Aggregators

In data analysis activities, it may be necessary to aggregate data items.

If a resulting aggregated item a belongs to a fuzzy set f, it is necessary to evaluate the membership degree of a to f. In [11], we started addressing the aggregation of membership degrees of elementary items, while in [12] we extended our vision to any kind of fields in elementary items. In both cases, the resulting aggregate item a must be provided with a membership degree. In this section, we report the both the linguistic and formal framework that we introduced in *J-CO-QL*$^+$ for user-defined fuzzy aggregators.



(a) Membership function for the fuzzy operator `casesPopulationFO`.



(b) Membership function for the fuzzy aggregator `integrateCasesFA`.

Fig. 3. Membership functions defined for aggregators in *J-CO-QL*$^+$.

Listing 2. *J-CO-QL*$^+$: Fuzzy Aggregator `integrateCasesFA`.

```
2. CREATE FUZZY AGGREGATOR integrateCasesFA
     PARAMETERS    cases  TYPE   ARRAY,
                   pop    TYPE   NUMERIC
     FOR ALL       cs     IN     cases
       AGGREGATE   cs     AS     av
     EVALUATE      100 * av / pop              // % of infected people
     POLYLINE      [ (0, 0.0), ( 1, 0.1),
                     (5, 0.8), (10, 1.0) ];
```

5.1 Basic Structure

In *J-CO-QL*$^+$ fuzzy aggregators are defined through the statement `CREATE FUZZY AGGREGATOR`. A first example can be found in Listing 2, while other examples are in Listing 3 and Listing 4. In comparison with the statement `CREATE FUZZY OPERATOR` (see Listing 1), the reader can see that the external structure is the same, i.e.:

- the clause `PARAMETERS` defines the formal parameters (for fuzzy aggregators, at least one parameter must be an array);
- the clause `PRECONDITION` is evaluated on parameters and must be satisfied in order to evaluate the aggregators;
- the clause `EVALUATE` evaluates the final numerical value that is matched against the membership function defined by the clause `POLYLINE` (if this clause is missing, the default polyline `[(0,0),(1,1)]` is considered), so as to return the membership degree.

The difference in comparison with the statement `CREATE FUZZY OPERATOR` is the presence, between the clause `PRECONDITION` and the clause `EVALUATE`, of a pool of specific clauses whose goal is to derived aggregated values to be combined within the clause `EVALUATE`.

We are ready to introduce the fuzzy aggregator `integrateCasesFA` defined in Listing 2. Its goal is to receive an array of integer values to aggregate: the array is named

cases, since each item denotes the number of cases of a disease; the second parameter is named pop (shortcut for "population"), because the aggregator must provide a membership degree based on the aggregated cases in relation with the population. In the following, we present the novel clauses.

- The clause FOR ALL scans all values in an array parameter, so as to perform aggregations. Specifically, all items cs in the array cases are explored.
- The sub-clause AGGREGATE evaluates, for each item cs in the array cases, an expression, whose value is aggregated into the "variable" specified after the keyword AS.
  Specifically, the expression to evaluate refers only to the item cs; this means that all the values in the array cases are aggregated (summed) into the final value av.
- The clause EVALUATE exploits the aggregated value av and the parameter pop. Specifically, it computes the percentage of cases over the number of inhabitants.
  The resulting value is matched against the polyline functions (depicted in Fig. 3b), so as to obtain the wished membership degree. The shape of the polyline function is defined in such a way it enhances degrees when the number of cases is greater than 1% of inhabitants.

5.2 Adding Local Derived Values

The fuzzy aggregator integrateCasesFA is a simple case of aggregator; in contrast, widely-used aggregators, such as the "Ordered Weighted Aggregator" (OWA, see [22]) are more complex to define and novel features must be provided by the language. Specifically, an OWA aggregation is a weighted aggregation, where weights are obtained through a monotone function; furthermore, since the greatest weights are used for the greatest values, the array of values must be suitably sorted. Thus, clauses to compute weights and sort arrays must be provided by the language.

To this end, consider the fuzzy aggregator named owaFA that is reported in Listing 3. The aggregator works on an array memberships of membership degrees, so as to aggregate them into a unique membership degree. To multiply items with the greatest values by the greatest weights, the array must be, at first, sorted in ascending order. As a monotone function, a parabolic function is used: $f(x) = x^2$. Weights are computed as follows: given the size s of the array memberships, denoting with $1 \leq i \leq s$ the position of each item in the array, the weight w_i of the i-th item is $w_i = f(i/s) - f((i-1)/s)$ Denoting with μ_i the i-th value in the array memberships, the final aggregated membership degree $\overline{\mu}$ is $\overline{\mu} = \Sigma_{u=1}^{s}(w_i \times \mu_i)$ (assuming that $f(1) - f(0) = 1$).

Hereafter, we present the sub-clauses exploited by the aggregator owaFA.

- The clause SORT generates a novel array, named sMemberships, sorting each numeric item ms in the array memberships in ascending order.
- The clause LOCALLY evaluates, for each item sms in the array sMemberships, a derived value that is used in the sub-clause AGGREGATE. The locally-derived value is named w, since it is the weight for each item in the sorted array sMemberships. Notice that special symbol POS, that denotes the actual position of the processed item in the array.

Listing 3. *J-CO-QL*$^+$: Fuzzy Aggregator `owaFA`.

```
3. CREATE FUZZY AGGREGATOR owaFA
     PARAMETERS   memberships  TYPE ARRAY
     SORT         ms           IN   memberships
       BY           ms         TYPE NUMERIC ASC  AS sMemberships
     FOR ALL      sms          IN   sMemberships
       LOCALLY      (POS^2 - (POS-1)^2) / (COUNT(sMemberships)^2) AS w
       AGGREGATE    sms * w    AS av
     EVALUATE     av;
```

Listing 4. *J-CO-QL*$^+$: Fuzzy Aggregator `weightedMembershipsFA`.

```
4. CREATE FUZZY AGGREGATOR weightedMembershipsFA
     PARAMETERS     memberships  TYPE   ARRAY,
                    w            TYPE   ARRAY
     PRECONDITION   COUNT (memberships) = COUNT (w)
     FOR ALL        m            IN     memberships
       LOCALLY      m * w[POS]   AS     wm
       AGGREGATE    wm           AS     am,
                    w [POS]      AS     aw
     EVALUATE       am / aw;
```

– finally, the clause `AGGREGATE`, for each item in the array multiplies its value by its weight and sums the results into the `av` value. Since we are obtaining an aggregated membership degree, the clause `POLYLINE` is not specified, so as to return the `av` value (that is a membership degree) as it is.

5.3 Multiple Aggregated Values

In the fuzzy aggregators so far presented, only one aggregated value is computed. However, more complex (and quite common) aggregators are based on more than one aggregated value. A typical example, is a "weighted aggregation", i.e., the array of items is accompanied by an array of weights. For example, if we have an array of memberships μ such as $s = |\mu|$ (size of the array μ) and a corresponding array of weights w, such that $|w| = s$. The final weighted aggregation of memberships $\overline{\mu}$ is $\overline{\mu} = (\Sigma_{i=1}^{s} \mu[i] \times w[i])/(\Sigma_{i_1}^{s} w[i])$. Clearly, both the numerator and the denominator are aggregated values.

Listing 4 reports the fuzzy aggregator named `weightedMembershipsFA`, which performs the above-presented aggregations. Hereafter, we describe it in details.

– The aggregator receives two parameters: `memberships` is the array of membership degrees to aggregate; `w` is the array of weights.
  Consequently, the clause `PRECONDITION` must verify that both array have the same size.
– Within the clause `FOR ALL`, the sub-clause `LOCALLY` computes the products of the single membership value with the corresponding weight (i.e., `w[POS]`).
  Then, the sub-clause `AGGREGATE` computes two aggregated values: `am` is the numerator, while `aw` is the denominator.
– Finally, the clause `EVALUATE` actually computes the final aggregated membership degree, by computing the fraction `am / aw`. Since this value is by definition in the range $[0, 1]$, no polyline function is defined and it is returned as it is.

5.4 Semantic Model

After we presented the statement provided by *J-CO-QL*$^+$ to define fuzzy aggregators, we report the semantic model behind, as presented in [10].

- A fuzzy aggregator is defined by means of a tuple

$$\langle LV, AV, evalExpr, Points, Params, Precond \rangle$$

  where LV is the (possibly empty) set of "local values", AV is the (non empty) set of "aggregated values", $evalExpr$ is the expression to evaluate with aggregated values, $Points$ is the array of points that constitute the polyline, $params$ is the list of parameters of the aggregator. *Precond* is the precondition: if it does not hold on *Params*, the aggregator is not evaluated.
- Each local value $lv_j \in LV$ is obtained as

$$lv_j = le_j(e, POS, Params)$$

  where le_j is the expression used to evaluate the local value; le_j can be seen as a function of an item e and of its position POS in the array, as well as of the list of parameters.
- An aggregated value $av_k \in AV$ is evaluated as

$$av_k = \Sigma_{e \in V} ae_k(e, pos(e), LV, Params)$$

  where ae_k is the expression used to evaluate the local value to aggregate; notice that ae_k depends on the item e, its position in the array, the set of local values and the list of parameters. The sum is performed for each item e in the V vector, where $V \in Params$.
- The final membership μ is obtained as

$$\mu = Polyline(Points, evalExpr(AV, Params))$$

  where the expression $evalExpr$ depends on the set AV of aggregated values and on the list of parameters.
  The resulting value passes through the function$Polyline$, which receives the list $Points$ of points specified in the clause `POLYLINE` (if missing, we assume that $Points = [(0, 0), (1, 1)]$).

The reader can see that, based on this semantic model, in *J-CO-QL*$^+$ it is possible to define a broad range of fuzzy aggregators. As far as we know, this is a novelty in the panorama of (soft) query languages.

In Sect. 6, we will show how to exploit fuzzy aggregators for soft querying data in *J-CO-QL*$^+$.

6 Case Study and Query

In this section, we show how to use fuzzy operators and fuzzy aggregators for soft querying a data set acquired from a Web portal.

6.1 Case Study

In this paper, we propose a case study that shows a possible application whose context were already considered in [8], i.e., acquiring and analyzing data concerned with the recent COVID-19 pandemic. Before going on, **it is worth clarifying** that the goal of the paper is only to show how it would be possible for non-programmers to perform the analysis task we considered with limited effort; we do not want at all to make any medical-political considerations in any way (which is outside the scope of this paper). Since the data set we used is publicly available on the Web, it is useful to show the final results.

Since the beginning of the COVID-19 pandemic, European countries have collected weekly data about the number of cases and the number of deaths due to COVID-19. The data set is publicly available on the web site of the European Centre for Disease Prevention and Control (ECDC)[1] and it is still updated[2] Clearly, this data set could provide interesting hints, if properly analyzed.

Consequently, we identified the following problem to address.

Problem 1. *Given the weekly cases of COVID-19 infections, observe the period from October 2020 (week* 40*) to March 2021 (week* 13*), so as to identify those countries that showed the highest amount of cases (per 100,000 inhabitants) and also possibly showed a significant amount of weekly peaks of cases.*

Problem 1 can be thought as a soft query.

Query 1. *Consider the universe of weekly cases records per each single country. In this universe, we consider the fuzzy set named* `ManyCasesRecords`*, which contains weekly cases records that are numerous (the greater the number of cases, the greater the membership degree). Consider now the universe of countries, on which two fuzzy sets are defined, named* `ManyCasesCountries` *and* `ManyPeaksCountries`*. The membership degree of a country to the fuzzy set* `ManyCasesCountries` *ranks countries on the bases of the aggregated recorded cases in the observed period (the greater the number of cases, the greater the membership degree). The membership degree of a country to the fuzzy set named* `ManyPeaksCountries` *ranks countries on the basis of the number of weekly peaks of cases (the greater the number of peaks, the greater the membership degree); the membership degree of a country to the fuzzy set* `ManyPeaksCountries` *is obtained by aggregating the membership degrees of weekly records to the fuzzy set* `ManyCasesRecords` *for that country, in such a way high membership degrees get the highest weights (OWA). We are interested in computing a third fuzzy set on the universe of countries, whose name is* `WantedCountries`*, in which the membership degrees ranks countries on the basis of a weighted aggregation of membership degrees to the fuzzy set* `ManyCasesCountries` *(with weight* 2*) and to the fuzzy set* `ManyPeaksCountries` *(with weight* 1*). Specifically, we are interested in countries that, in the observed period, have a membership degree to the fuzzy set* `WantedCountries` *that is no less than* 0.80*.*

[1] ECDC web site: https://www.ecdc.europa.eu/en/covid-19 accessed on 12/04/2024.

[2] Data set: https://opendata.ecdc.europa.eu/covid19/nationalcasedeath/json/ accessed on 12/04/2024.

Listing 5. *J-CO-QL*$^+$: Java Functions `getYear` and `getWeek`.

```
5. CREATE JAVA FUNCTION getYear
     PARAMETERS yearWeek     TYPE String
     CLASS      GetYearClass
     CLASS BODY {
       // yearWeek : is supposed to be in the form "YYYY-WW"
       public static int getYear(String yearWeek)  {
         try { return Integer.parseInt(yearWeek.substring(0,4)); }
         catch (Exception e) { return 0; }
       }
     } END BODY;

6. CREATE JAVA FUNCTION getWeek
     PARAMETERS yearWeek     TYPE String
     CLASS      GetWeekClass
     CLASS BODY {
       // yearWeek : is supposed to be in the form "YYYY-WW"
       public static int getWeek(String yearWeek)  {
         try { return Integer.parseInt(yearWeek.substring(5,7)); }
         catch (Exception e) { return 0; }
       }
     } END BODY;
```

Clearly, in order to evaluate the memberships of countries to the fuzzy sets, it is necessary to aggregate records of weekly cases.

6.2 User-Defined Functions

The *J-CO-QL*$^+$ script that implements Query 1 has to process string fields that denote the week for which cases are recorded in the form `"YYYY-WW"`, where `YYYY` denotes the year (four digits) and `WW` denotes the number of week (two digits) in the year. A couple of functions to manipulate these strings and extract year and week number are necessary.

As extensively presented in [12], *J-CO-QL*$^+$ provides constructs to define either Java functions or JavaScript functions, i.e., user-defined functions that are implemented either in the Java language or in the JavaScript language. Listing 5 reports the definition of the functions named `getYear` and `getWeek`, implemented in the Java language The interested reader can refer to [12] for details: here, we simply note that the two functions receive one formal parameter named `yearWeek` and return, respectively, the number corresponding to the year and the number corresponding to the week.

User that are more familiar with the JavaScript language can use this language; Listing 6 reports the JavaScript implementation of the same user-defined functions presented in Listing 5. Simply notice that the execution of JavaScript functions is significantly slower than the execution of Java functions.

Listing 6. *J-CO-QL*$^+$: Alternative Javascript Functions `getYear` and `getWeek`.

```
5 alt. CREATE JAVASCRIPT FUNCTION getYear
         PARAMETERS yearWeek     TYPE String
         BODY{
           // yearWeek: is supposed to be in the form "YYYY-WW"
           if (yearWeek == null)
             return 0;
           var y = parseInt (yearWeek.substring (0,4), 10);
           return y;
         } END BODY;

6 alt. CREATE JAVASCRIPT FUNCTION getWeek
         PARAMETERS yearWeek     TYPE String
         BODY{
           // yearWeek: is supposed to be in the form "YYYY-WW"
           if (yearWeek == null)
             return 0;
           var w = parseInt (yearWeek.substring (5,7), 10);
           return w;
         } END BODY;
```

6.3 Query

Before we present the *J-CO-QL*$^+$ query for the case study, it is worth introducing shortly the data and execution models of the language (readers can find details in [6,20]).

- A *JSON* document is the basic computational unit. *JSON* documents are grouped within "collections": an instruction takes one or two collections as input and generates a new collection.
- A query (i.e., script) in *J-CO-QL*$^+$ is a sequence of instructions. They constitute a "piped flow", in such a way an instruction receives a "temporary collection" (generated by the previous instruction) as input and possibly generates a novel temporary collection as output. The "temporary" adjective denotes that the collection is not saved in any database, but is a temporary result of the process. Instructions can also acquire data either from *JSON* databases or from Web sources.

Listings 7 and 8 actually perform Query 1 (we divide the script in two parts for editorial issues); notice that, the script in Listing 7 starts from the line 7, because the definitions of the fuzzy operator, the three fuzzy aggregators and the two Java functions reported in previous listings are part of the query itself. Hereafter, we present how the script works.

Acquiring Data. On Line 7, the instruction `GET COLLECTION FROM WEB` retrieves the starting collection from the ECDC web site.

The collection is actually composed by one single *JSON* document and Fig. 4 shows an excerpt of it. Notice that the document, apart from meta-information fields such as `url` and `timestamp`, holds an array field (named `data`) that contains sub-documents, such that each contained-document reports a weekly record for a single country. We shortly present some fields, leaving out uninteresting fields for this

case study: the field `country` reports the European country to which the weekly record is referred to; the field `population` reports the population of the country; the field `year_week` reports the referring week of the of the record; finally, the field `indicator` can assume only two values, `"cases"` and `"deaths"`, in order to specify that the value reported in the field `weekly_count` is referred, respectively, to the number of COVID-19 cases or to the number of COVID-19 deaths. Furthermore, notice that sub-documents in the array field `data` may not have always the same structure, as shown by the two sub-documents structures highlighted by yellow boxes in Fig. 4. For the sake of completeness, the array field `data` contains $12,648$ weekly records sub-documents.

Listing 7. *J-CO-QL*$^+$: Script 1.

```
7.  GET COLLECTION FROM WEB
      "https://opendata.ecdc.europa.eu/covid19/nationalcasedeath/json";

8.  EXPAND
      UNPACK   WITH ARRAY .data
      ARRAY    .data  TO   .unpackedData;

9.  FILTER
      CASE WHERE WITH .unpackedData.item.weekly_count
             AND .unpackedData.item.indicator = "cases"
        GENERATE
          BUILD {
            .country     : .unpackedData.item.country,
            .population  : .unpackedData.item.population,
            .weeklyCases : .unpackedData.item.weekly_count,
            .year        : getYear(.unpackedData.item.year_week),
            .week        : getWeek(.unpackedData.item.year_week)
          };

10. FILTER
      CASE WHERE (.year = 2020 AND .week >= 40)
              OR(.year = 2021 AND .week <= 13)
        GENERATE
          CHECK FOR
            FUZZY SET ManyCasesRecords
              USING casesPopulationFO(.weeklyCases, .population);

11. GROUP
      PARTITION  KNOWN FUZZY SETS ManyCasesRecords
             AND WITH   .country, .population,
                        .weeklyCases, .week, .year
        BY .country, .population INTO .countryData
        DROP GROUPING FIELDS;
```

Preparing Data. The role of the instruction `EXPAND` on Line 8 of Listing 7 is to unnest all the *JSON* sub-documents in the field `data` of the lonely document in the temporary collection depicted in Fig. 4; Fig. 5 depicts one unnested document, specifically the sub-document surrounded by the red-dashed box in Fig. 4, related to the COVID-19 cases in Luxembourg during the week 51 of 2020. From Fig. 5, notice how the instruction `EXPAND` works: for each document within the array field `data`, a novel document is generated into the output temporary collection; in this document (see Fig. 5), the novel field `unpackedData`) is further structured as a document with two fields, where

Fig. 4. Excerpt of document collection retrieved by instruction 7.

the field `item` contains the unnested document, while the field `position` denotes the position occupied by the unnested document in the original array. Thus, after the execution of the instruction `EXPAND`, the generated temporary collection contains $12,648$ documents reporting both weekly records about COVID-19 cases and weekly records about COVID-19 deaths.

The role of the instruction `FILTER` on Line 9 of Listing 7 is to select only documents that describe weekly records related to COVID-19 cases from the current temporary collection. The instruction also restructures documents in order to be easily managed afterwards.

Hereafter, we explain it in details. For each document in the current temporary collection, the clause `CASE WHERE` selects only those documents related to

```
{
  "url" : "https://opendata.ecdc.europa.eu/covid19/nationalcasedeath/json/",
  "timestamp"     : "2024-04-12T17:30:47.110",
  "unpackedData" : {
     "position"      : 8211,
     "item"          : {
       "continent"          : "Europe",
       "country"            : "Luxembourg",
       "country_code"       : "LUX",
       "cumulative_count"   : 37094,
       "indicator"          : "cases",
       "note"               : "",
       "population"         : 645397,
       "rate_14_day"        : 908.588,
       "source"             : "TESSy COVID-19",
       "weekly_count"       : 2623,
       "year_week"          : "2020-51"
     }
  }
}
```


Fig. 5. Example of document in collection provided by instruction 8.

weekly records of COVID-19 cases that have been actually recorded (the field `expandedData.item.weekly_count` must be present), while the other documents are discarded. The remainder of the instruction works on the selected documents: for each of them, the following section `GENERATE` generates a novel document, by means of the block `BUILD`, where only the fields named `country`, `population` and `weekly_count` (renamed as `weeklyCases`) are reported at root-level. Moreover, two novel fields, named `year` and `week`, are appended by exploiting, respectively, the Java functions `getYear` and `getWeek`, previously defined in Listing 5.

Figure 6 shows how the document depicted in Fig. 5 is selected and transformed by the `FILTER` instruction. After the execution of the `FILTER` instruction on Line 9, the current temporary collection contains 6,016 documents.


```
{
  "country"       : "Luxembourg",
  "population"    : 645397,
  "week"          : 51,
  "weeklyCases"   : 2623,
  "year"          : 2020
}
```


Fig. 6. Example of document in collection provided by instruction 9.

The role of the following `FILTER` instruction on Line 10 of Listing 5 is to select, from the temporary collection, only those weekly records related to COVID-19 cases in the period of interest.

Hereafter, we explain the instruction in details. The clause `CASE WHERE` selects only those documents that describe cases recorded in the period from October 2020 (week 40) to March 2021 (week 13). For each selected document, the following

clause GENERATE leaves the document unchanged. Furthermore, it "fuzzifies" the document through the clause CHECK FOR is used: the branch FUZZY SET evaluates the membership degree of the document to the fuzzy set ManyCasesRecords; the USING sub-clause provides the soft condition, that is based on the fuzzy operator casesPopulationFO, previously defined in Listing 1; the operator is called by passing (as actual parameters) the values of the fields weeklyCases and population, so as to compute a membership degree. The fuzzification of the document implies that the special field ~fuzzysets is added at root-level: this field contains a field named ManyCasesRecords, whose value represents the membership degree to the fuzzy set.

Figure 7 shows how the document depicted in Fig. 6 is selected and fuzzified by the FILTER instruction on Line 10: notice the presence of the newly added special field ~fuzzysets, highlighted in the red box, holding inside the field ManyCasesRecords with the membership degree of the eponym fuzzy set.

After the execution of the instruction FILTER on Line 10, the current temporary collection contains 837 documents.


```
{
  "country"       : "Luxembourg",
  "population"    : 645397,
  "week"          : 51,
  "weeklyCases"   : 2623,
  "year"          : 2020,
  "~fuzzysets"    : {
    "ManyCasesRecords"   : 0.917736066341084
  }
}
```


Fig. 7. Example of document in collection provided by instruction 10.


```
{
  "country"       : "Luxembourg",
  "population"    : 645397,
  "countryData"   : [
    { … },                                  // other sub document
    …,
    {
      "week"          : 51,
      "weeklyCases"   : 2623,
      "year"          : 2020,
      "~fuzzysets"    : {
        "ManyCasesRecords"   : 0.917736066341084
      }
    },
    …,
    { … }                                   // other sub document
  ]
}
```


Fig. 8. Example of document in collection provided by instruction 11.

The role of the GROUP instruction on Line 11 of Listing 5 is to group together all the weekly records in the temporary collection related to the same country.

Hereafter, we explain it in details. The clause `PARTITION` selects all those documents that belong to the fuzzy set `ManyCasesRecords` and have the fields `country`, `population`, `weeklyCases`, `year` and `week`. The clause `BY` creates, for all documents with the same value in the fields `country` and `population`, one single novel document with the fields `country` and `population` at root-level; furthermore, as specified by the clause `INTO`, an array field named `countryData`, which contains all source documents grouped together (as specified by the option `DROP GROUPING FIELDS`, from these documents the grouping fields `country` and `population` are removed).

Figure 8 shows an example of documents generated by the instruction `GROUP`. Notice how the document depicted in Fig. 7 is now included within the array field `countryData` (highlighted with a yellow box). After the execution of the `GROUP` instruction, the current temporary collection contains 31 documents, one for each country of the European Union plus a few others of particular interest like Norway.

Listing 8. *J-CO-QL*$^+$: Script 2.

```
12.  FILTER
       CASE WHERE WITH .countryData
         GENERATE
           CHECK FOR
             FUZZY SET ManyCasesCountries
               USING integrateCasesFA (EXTRACT_ARRAY(.weeklyCases
                                 FROM ARRAY .countryData), .population),
             FUZZY SET ManyPeaksCountries
               USING owaFA (MEMBERSHIP_ARRAY (ManyCasesRecords
                                 FROM ARRAY .countryData)),
             FUZZY SET WantedCountries
               USING weightedMembershipsFA (MEMBERSHIP_ARRAY(
                    [ManyCasesCountries, ManyPeaksCountries]), [2, 1]);

13.  FILTER
       CASE WHERE KNOWN FUZZY SETS WantedCountries
         GENERATE
           ALPHACUT 0.80 ON WantedCountries
           BUILD {
             .country     : .country,
             .population  : .population,
             .ranking     : MEMBERSHIP_TO (WantedCountries)
           }
           DEFUZZIFY;

14.  USE DB Webist2024
       ON SERVER MongoDB 'http://127.0.0.1:27017';

15.  SAVE AS ecdcCovidCasesAnalysis@Webist2024;
```

Soft Querying with Fuzzy Aggregators. The instruction `FILTER` on Line 12 of Listing 8 actually performs the soft query. Hereafter, we explain it in details.

The clause `CASE WHERE` selects those documents that have the field `countryData`. The block `GENERATE` leaves the documents unchanged, apart from the fact that it evaluates their belonging to fuzzy sets. This time the clause `CHECK FOR` contains three different branches, each one devoted to compute the membership degree to a specific fuzzy set:

- The first branch `FUZZY SET` evaluates the membership to the fuzzy set `ManyCasesCountries`. To do this, the soft condition specified after the `USING` keyword exploits the fuzzy aggregator `integrateCasesFA`, defined in Listing 2. To call the fuzzy aggregator, an array of numbers must be provided as first actual parameter: since the array field `countryData` contains nested documents, the built-in function `EXTRACT_ARRAY` creates a novel array of numbers by projecting the array field `countryData` on the inner (numerical) field `weeklyCases`. The second actual parameter for the fuzzy aggregator is the field `population`.
  The evaluation of this first fuzzy set implies adding, at root level, the special field `~fuzzysets`; within it, the field `ManyCasesCountries` reports the membership degree.
- The second branch `FUZZY SET` evaluates the membership of the current document to the fuzzy set `ManyPeaksCountries`, through the fuzzy aggregator `owaFA` defined in Listing 3.
  Apart from the fact that the aggregator `owaFA` performs an OWA aggregation (instead of a cumulative aggregation), the branch behaves similarly to the previous one: this time, the aggregator is called by passing an "array of memberships" obtained by the built-in function `MEMBERSHIP_ARRAY`, which projects the array `countryData` on the fuzzy set `ManyCasesRecords` contained in the inner special field `~fuzzysets`.
  After the evaluation of this second fuzzy set, the field `ManyPeaksCountries` is added to the special field `~fuzzysets` at root-level.
- The third branch `FUZZY SET` evaluates the degree of the current document to the fuzzy set `WantedCountries`. The goal is to combine the memberships to the fuzzy sets `ManyCasesCountries` and `ManyPeaksCountries` in such a way the former contributes with weight 2, while the latter contributes with weight 1.
  To perform this task, the fuzzy aggregator `weightedMembershipsFA` (reported in Listing 4) is exploited. This time, the "array of memberships", needed as first actual parameter, is retrieved by exploiting a different behaviour of the built-in function `MEMBERSHIP_ARRAY`: the function creates a novel array by taking the previously-calculated memberships to the listed fuzzy sets (i.e., `ManyCasesCountries` and `ManyPeaksCountries`), whose values are taken from the special field `~fuzzysets`. The second actual parameter is a constant array that reports the weights to apply (i.e., 2 and 1, respectively); this way, requirements in Query 1 are met.
  The third and final field is added to the special field `~fuzzysets`, with the membership to the fuzzy set `WantedCountries`.

Figure 9 shows the document presented in Fig. 8 after the execution of the `FILTER` instruction on Line 12. Notice the special field `~fuzzysets`, with the membership degrees to the fuzzy sets `ManyCasesCountries` `ManyPeaksCountries` and `WantedCountries` that are highlighted by a blue box.

After the execution of the instruction `FILTER` on Line 12, the temporary collection still contains 31 documents. Table 1 reports, for each document/country, the fields `country` and `population`, as well as the membership degrees to the three fuzzy sets `ManyCasesCountries` `ManyPeaksCountries` and `WantedCountries`.

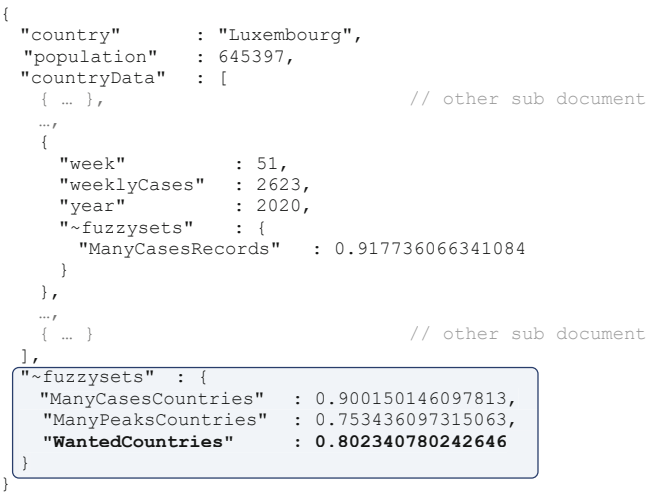


Fig. 9. Example of document in collection provided by instruction 12.

At this point, we want to repeat the disclaimer we made at the beginning of Sect. 6.1: any medical/political consideration is out of the scope of this paper.

The instruction `FILTER` on Line 13 of Line 12 of Listing 8 concludes the soft query.

The clause `CASE WHERE` selects only those documents that belong to the fuzzy set `WantedCountries`.

In the section `GENERATE`, the clause `ALPHACUT` discards those documents whose membership to the fuzzy set `WantedCountries` is less than 0.80. This way, only documents that describe countries that actually had a high number of COVID-19 cases, together with many peaks of weekly COVID-19 cases during the monitored period (or very close to this situation) are selected.

The final block `BUILD` restructures the output documents and the option `DEFUZZIFY` removes the special filed `~fuzzysets`, so as documents become again classical crisp *JSON* documents.

Figure 10 shows how the document shown in Fig. 9 is selected and restructured by the instruction `FILTER` on Line 13 of Listing 8.

After the execution of the instruction `FILTER` on Line 13, the temporary collection now contains 7 documents, i.e., those corresponding to the rows above the thick line in Table 1.

Saving the Results. The instruction `USE DB` on Line 14 of Listing 8 specifies the database to connect with. Specifically, notice that the database `Webist2024` is managed by *MongoDB*.

Table 1. Tabular representation of the temporary collection generated by Line 12 of Listing 8.

country	population	ManyCasesCountries	ManyPeaksCountries	WantedCountries
Czechia	10,516,707	1.00	0.95	0.97
Slovakia	5,434,712	1.00	0.92	0.95
Slovenia	2,107,180	1.00	0.90	0.93
Sweden	10,452,326	0.88	0.82	0.84
Estonia	1,331,796	0.90	0.78	0.82
Luxembourg	645,397	0.90	0.75	0.80
Netherlands	17,590,672	0.87	0.77	0.80
Portugal	10,352,042	0.88	0.74	0.79
Lithuania	2,805,998	0.90	0.72	0.78
Hungary	9,689,010	0.87	0.73	0.78
Croatia	3,862,305	0.86	0.71	0.76
Liechtenstein	39,308	0.86	0.69	0.75
Poland	37,654,247	0.85	0.67	0.73
Latvia	1,875,757	0.82	0.68	0.72
Italy	59,030,133	0.83	0.66	0.71
Belgium	11,617,623	0.87	0.63	0.71
Cyprus	904,705	0.81	0.65	0.71
France	67,871,925	0.84	0.64	0.71
EU/EEA (total)	452,576,117	0.82	0.62	0.69
Austria	8,978,929	0.83	0.61	0.68
Spain	47,432,893	0.82	0.61	0.68
Bulgaria	6,838,937	0.78	0.61	0.67
Malta	520,971	0.80	0.57	0.65
Romania	19,042,455	0.71	0.52	0.58
Ireland	5,060,004	0.63	0.37	0.46
Denmark	5,873,420	0.45	0.30	0.35
Germany	83,237,124	0.47	0.28	0.35
Greece	10,459,782	0.37	0.24	0.28
Norway	5,425,270	0.20	0.08	0.12
Finland	5,548,241	0.15	0.06	0.09
Iceland	376,248	0.10	0.08	0.08

```
{
  "country"       : "Luxembourg",
  "population"    : 645397,
  "ranking"       : 0.802340780242646
}
```

Fig. 10. Example of document saved in the collection `ecdcCovidCasesAnalysis` by instruction 15.

Finally, the resulting collection, which contains documents describing countries satisfying the soft query formalized by Query 1, is finally saved by the instruction `SAVE AS` on Line 15, into the database `Webist2024` with the name `ecdcCovidCasesAnalysis`.

7 Conclusions

In this paper, we present the potential application of *Soft Web Intelligence* for acquiring and analyzing *JSON* data sets from Web sources, by exploiting soft-querying capabilities provided by fuzzy operators and fuzzy aggregators within the *J-CO-QL*$^+$ language. We showed that users can define their own operators and aggregators, to deal with possibly complex situations in which ranking capabilities are necessary to deal with imprecise contexts.

The paper resumes the vision of *Soft Web Intelligence*; then, it briefly describes the *J-CO* Framework and how fuzzy sets are basically supported by *J-CO-QL*$^+$. The concept of fuzzy aggregator is specifically presented (it is the novel contribution of our current research) in a dedicated section. Finally, a practical case study that exploits a real data set about records of COVID-19 cases is presented, to show the practical application of the concepts and illustrate the potential use of *J-CO-QL*$^+$ for non programmers in *Soft Web Intelligence* tasks.

As a future work, we will investigate the unification of fuzzy operators and fuzzy aggregator in one single concept, so as to provide a cleaner semantic model. We will also investigate the integration of aggregation within the novel concept, recently introduced in *J-CO-QL*$^+$ [13], of "multi-grade models for fuzzy sets", i.e., models of fuzzy sets in which multiple degrees characterize the belonging of an item to a fuzzy set.

The framework is available on a Github page[3].

References

1. Alahakoon, D., Yu, X.: Smart electricity meter data intelligence for future energy systems: a survey. IEEE Trans. Ind. Inf. **12**(1), 425–436 (2015)
2. Garcia Bringas, P., Pastor, I., Psaila, G.: Can blockchain technology provide information systems with trusted database? The case of hyperledger fabric. In: Cuzzocrea, A., Greco, S., Larsen, H.L., Saccà, D., Andreasen, T., Christiansen, H. (eds.) FQAS 2019. LNCS (LNAI), vol. 11529, pp. 265–277. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-27629-4_25
3. Dombi, J., Jónás, T.: Weighted aggregation systems and an expectation level-based weighting and scoring procedure. Eur. J. Oper. Res. **299**(2), 580–588 (2022)
4. Fosci, P., Marrara, S., Psaila, G.: Geosoft: a language for soft querying features within geojson information layers. In: International Conference on Web Information Systems and Technologies, pp. 196–219. Springer, Cham (2020)
5. Fosci, P., Psaila, G.: J-co, a framework for fuzzy querying collections of json documents. In: Flexible Query Answering Systems: 14th International Conference, FQAS 2021, Bratislava, Slovakia, 19–24 September 2021, Proceedings 14, pp. 142–153. Springer, Cham (2021)
6. Fosci, P., Psaila, G.: Towards flexible retrieval, integration and analysis of json data sets through fuzzy sets: a case study. Information **12**(7), 258 (2021)
7. Fosci, P., Psaila, G.: Soft integration of geo-tagged data sets in J-CO-QL+. ISPRS Int. J. Geo Inf. **11**(9), 484 (2022)
8. Fosci, P., Psaila, G.: Soft web intelligence with the j-co framework. In: International Conference on Web Information Systems and Technologies, pp. 142–165. Springer, Cham (2022)

[3] Github repository of the *J-CO* Framework: https://github.com/JcoProjectTeam/JcoProjectPage, accessed on 12/04/2024.

9. Fosci, P., Psaila, G.: Towards soft web intelligence by collecting and processing json data sets from web sources. In: Proceedings of the 18th International Conference on Web Information Systems and Technologies (2022)
10. Fosci, P., Psaila, G.: Enhancing soft web intelligence with user-defined fuzzy aggregators. In: WEBIST, vol. 1, pp. 258–267. Science and Technology Publications, Lda (2023)
11. Fosci, P., Psaila, G.: Fuzzy aggregators in practice: meta-model and implementation. In: International Conference on Soft Computing Models in Industrial and Environmental Applications, pp. 56–68. Springer, Cham (2023)
12. Fosci, P., Psaila, G.: Soft querying powered by user-defined functions in J-CO-QL+. Neurocomputing **546**, 126311 (2023)
13. Fosci, P., Psaila, G.: A unified view of multi-grade fuzzy-set models in J-CO-QL+. Neurocomputing **565**, 126968 (2024)
14. Han, J., Chang, K.C.: Data mining for web intelligence. Computer **35**(11), 64–70 (2002)
15. Kacprzyk, J., Zadrożny, S.: Soft computing and web intelligence for supporting consensus reaching. Soft. Comput. **14**(8), 833–846 (2010)
16. Li, H., Yen, V.C.: Fuzzy Sets and Fuzzy Decision-Making. CRC Press (1995)
17. Negash, S., Gray, P.: Business intelligence. In: Handbook on Decision Support Systems 2, pp. 175–193. Springer, Cham (2008)
18. Poli, V.S.R.: Fuzzy data mining and web intelligence. In: International Conference on Fuzzy Theory and Its Applications (iFUZZY), pp. 74–79. IEEE (2015)
19. Psaila, G., Fosci, P.: Toward an anayist-oriented polystore framework for processing json geo-data. In: International Conference on Applied Computing 2018, Budapest; Hungary, 21–23 October 2018, pp. 213–222. IADIS (2018)
20. Psaila, G., Fosci, P.: J-co: a platform-independent framework for managing geo-referenced json data sets. Electronics **10**(5), 621 (2021)
21. Reddy, P.: Fuzzyalgol: fuzzy algorithmic language for designing fuzzy algorithms. J. Comput. Sci. Eng. **2**(2), 21–24 (2010)
22. Yager, R.R.: On ordered weighted averaging aggregation operators in multicriteria decision making. IEEE Trans. Syst. Man Cybern. **18**(1), 183–190 (1988)
23. Yao, Y., Zhong, N., Liu, J., Ohsuga, S.: Web intelligence (WI) research challenges and trends in the new information age. In: Asia-Pacific Conference on Web Intelligence, pp. 1–17. Springer, Cham (2001)
24. Zadeh, L.A.: Fuzzy sets. Inf. Control **8**(3), 338–353 (1965)
25. Zadeh, L.A.: A note on web intelligence, world knowledge and fuzzy logic. Data Knowl. Eng. **50**(3), 291–304 (2004)
26. Zadeh, L.A.: Web intelligence, world knowledge and fuzzy logic–the concept of web IQ (WIQ). In: International Conference on Knowledge-Based and Intelligent Information and Engineering Systems, pp. 1–5. Springer, Cham (2004)
27. Zhang, Y.Q., Lin, T.Y.: Computational web intelligence (CWI): synergy of computational intelligence and web technology. In: World Congress on Computational Intelligence, vol. 2, pp. 1104–1107. IEEE (2002)
28. Zhong, N., et al.: Web intelligence meets brain informatics. In: International Workshop on Web Intelligence Meets Brain Informatics, pp. 1–31. Springer, Cham (2006)
29. Zuccala, A., Thelwall, M., Oppenheim, C., Dhiensa, R.: Web intelligence analyses of digital libraries: a case study of the national electronic library for health (NeLH). J. Documentation (2007)